

Cours 22 — Le système d'exploitation

Semaine 22 ■ Manuel Hachette ch. 1 (§ 1.5, 1.6) et ch. 12 (§ 12.1) ■ 2×1 h

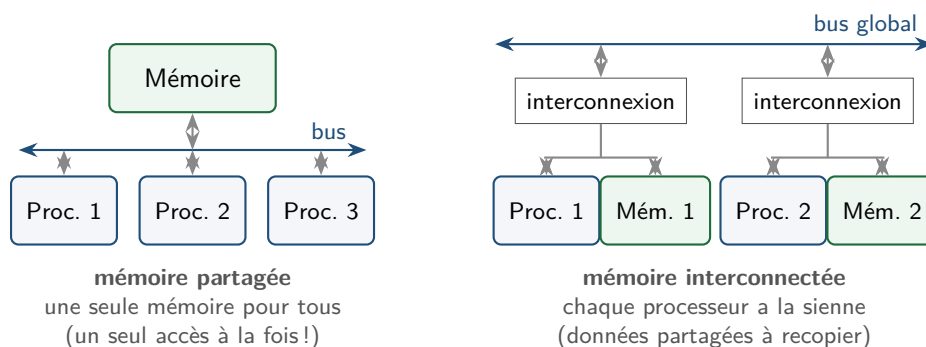
Objectifs

- Situer les entrées-sorties et les architectures multiprocesseurs
- Expliquer les rôles d'un système d'exploitation
- Distinguer logiciels libres et propriétaires

1. Fin du voyage matériel

Entrées-sorties. Clavier, écran, disque, carte réseau... sont des mondes lents comparés au processeur (un disque répond en millisecondes : des *millions* de cycles). Le processeur ne les attend pas au guichet : les périphériques signalent leurs événements (**interruptions**) et le processeur fait autre chose entre-temps.

Multiprocesseurs. Depuis les années 2000, la fréquence plafonne (chaleur!) ; on multiplie donc les **cœurs** — plusieurs processeurs sur la même puce. Votre téléphone en a 8. Le manuel (ch. 1, § 6) distingue deux familles d'architectures :



Chaque architecture a ses avantages selon le programme : la mémoire partagée est simple mais un seul processeur y accède à la fois ; la mémoire interconnectée évite l'embouteillage mais oblige à recopier les données partagées via le bus global. Nos programmes Python n'utilisent de toute façon qu'un cœur : paralléliser un algorithme est un art difficile (aperçu en Terminale).

2. Le système d'exploitation, chef d'orchestre

Entre le matériel et vos applications, un logiciel permanent : le **système d'exploitation** (OS — Linux, Windows, macOS, Android, iOS). Ses grandes missions :

- **gérer les processus** : des dizaines de programmes « en même temps » sur quelques cœurs — l'OS distribue le processeur par tranches de temps ;
- **gérer la mémoire** : chaque processus reçoit sa zone, isolée de celles des autres (un programme qui plante n'emporte pas les voisins) ;
- **gérer les fichiers** : organiser les octets du disque en fichiers et dossiers (semaine prochaine) ;
- **gérer les périphériques** : parler à chaque matériel via son **pilote** ;
- **gérer les utilisateurs** : comptes, mots de passe, **droits** (semaine prochaine aussi).

Nos programmes ne touchent jamais le matériel directement : **open**, **print**, **input** sont des **appels au système**.

3. Libre ou propriétaire ?

Un logiciel est **libre** si sa licence garantit quatre libertés : **utiliser**, **étudier** (code source disponible), **redistribuer**, **modifier**. Sinon il est **propriétaire** (code fermé, usage sous conditions).

- **GNU/Linux** : libre, omniprésent côté serveurs (Internet tourne dessus), à la base d'Android ; des distributions comme Debian ou Ubuntu.
- Windows et macOS : propriétaires.
- Libre $\neq$ gratuit : c'est une question de **droits**, pas de prix — enjeu de souveraineté, de sécurité (code auditable), d'éducation.

Python, Firefox, VLC, Wikipédia (contenu) : l'écosystème libre est déjà votre quotidien.

4. Parler à l'OS : la ligne de commande

Avant les fenêtres, on dialoguait avec l'OS par le **terminal** — et les professionnels le font toujours : plus précis, scriptable, présent sur tous les serveurs. C'est l'objet du TP :

```
$ pwd          où suis-je ?  
$ ls           que contient ce dossier ?  
$ cd Documents se déplacer  
$ mkdir NSI    créer un dossier
```

À retenir

- Périphériques lents $\rightarrow$ interruptions ; fréquence plafonnée $\rightarrow$ multi-cœurs.
- OS : processus, mémoire, fichiers, périphériques, utilisateurs — les programmes passent par lui (appels système).
- Libre = utiliser, étudier, redistribuer, modifier ; Linux domine les serveurs ; libre $\neq$ gratuit.
- Le terminal : dialogue direct avec l'OS — pwd, ls, cd, mkdir...